CS 301
High-Performance Computing

Assignment 04 - Particle Interpolation and Mover Performance Analysis

Visha Sitapara (202301414)
Dhruv Patel (202301095)

March 11, 2026

Contents

1 Introduction **3**

2 Hardware Details **3**
2.1 Hardware Details for LAB207 PCs 3
2.2 Hardware Details for HPC Cluster (Node gics0) 4

3 Problem Description **5**

4 Benchmarking Methodology **5**
4.1 Experiment 01: Scaling with Number of Particles 5
4.2 Experiment 02: Consistency Across Configurations 5
4.3 Experiment 03: The Mover Operation and OpenMP Parallelization 5

5 Graphical Results **6**
5.1 Experiment 01 Plots 6
5.2 Experiment 02 Plots 7
5.3 Experiment 03 Plots 8

6 Implementation Approach **9**
6.1 Schematic Flow Diagram 9

7 Analysis and Questions **9**

8 Conclusions **10**

1 Introduction

This assignment builds upon the interpolation algorithm from Assignment 03. We extend the scattered point to mesh interpolation scheme by analyzing its performance under various conditions and introducing a new particle tracking phase known as the Mover. The final phase requires parallelizing this Mover operation using basic OpenMP directives. All particles and grid points are constrained within a 1×1 domain.

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Address sizes: 46 bits physical, 48 bits virtual
- Byte Order: Little Endian
- CPU(s): 12
- On-line CPU(s) list: 0-11
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 151
- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- Stepping: 5
- CPU max MHz: 4600.0000
- CPU min MHz: 800.0000
- BogoMIPS: 5990.40
- Virtualization: VT-x
- L1d cache: 288K
- L1i cache: 192K

- L2 cache: 7.5M
- L3 cache: 18M
- NUMA node0 CPU(s): 0-11
- Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm epb invpcid_single tpr_shadow vnmi flexpriority ept vpid fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln pts

2.2 Hardware Details for HPC Cluster (Node gics0)

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 24
- On-line CPU(s) list: 0-23
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 2
- NUMA node(s): 2
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 63
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- Stepping: 2
- CPU MHz: 2907.656
- BogoMIPS: 4804.69
- Virtualization: VT-x
- L1d cache: 32K
- L1i cache: 32K

- L2 cache: 256K
- L3 cache: 15360K
- NUMA node0 CPU(s): 0-5, 12-17
- NUMA node1 CPU(s): 6-11, 18-23

3 Problem Description

The objective is to analyze the performance of interpolating scattered points onto a structured mesh, and then updating particle positions using a Mover operation.

During the Mover phase, each particle is displaced randomly within $+\Delta x$ and $\pm\Delta y$, where $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$. If a move attempts to place a particle outside the $(0.0 \leq x, y \leq 1.0)$ domain, the displacement is regenerated until the updated position remains strictly inside the boundaries.

4 Benchmarking Methodology

We conduct three specific experiments:

4.1 Experiment 01: Scaling with Number of Particles

We observe how interpolation execution time scales with the number of particles using three grid configurations:

1. $N_x = 250$, $N_y = 100$, points range $= 10^2, 10^4, 10^6, 10^8, 10^9$, Maxiter $= 10$.
2. $N_x = 500$, $N_y = 200$, points range $= 10^2, 10^4, 10^6, 10^8, 10^9$, Maxiter $= 10$.
3. $N_x = 1000$, $N_y = 400$, points range $= 10^2, 10^4, 10^6, 10^8, 10^9$, Maxiter $= 10$.

For each count, particles are initialized, interpolation is executed for Maxiter times, and the total interpolation time is accumulated (excluding memory allocation).

4.2 Experiment 02: Consistency Across Configurations

Using the three configurations above, the number of particles is fixed to 10^8. Particles are initialized only once. Interpolation is executed for Maxiter $= 10$ iterations, and total interpolation time is accumulated to evaluate performance consistency.

4.3 Experiment 03: The Mover Operation and OpenMP Parallelization

We fix parameters as $N_x = 1000$, $N_y = 400$, points $= 14$ million, and Maxiter $= 10$. In each iteration, interpolation is followed by the Mover operation. The Mover portion is then parallelized using OpenMP with a fixed thread count of 4.

5 Graphical Results

5.1 Experiment 01 Plots

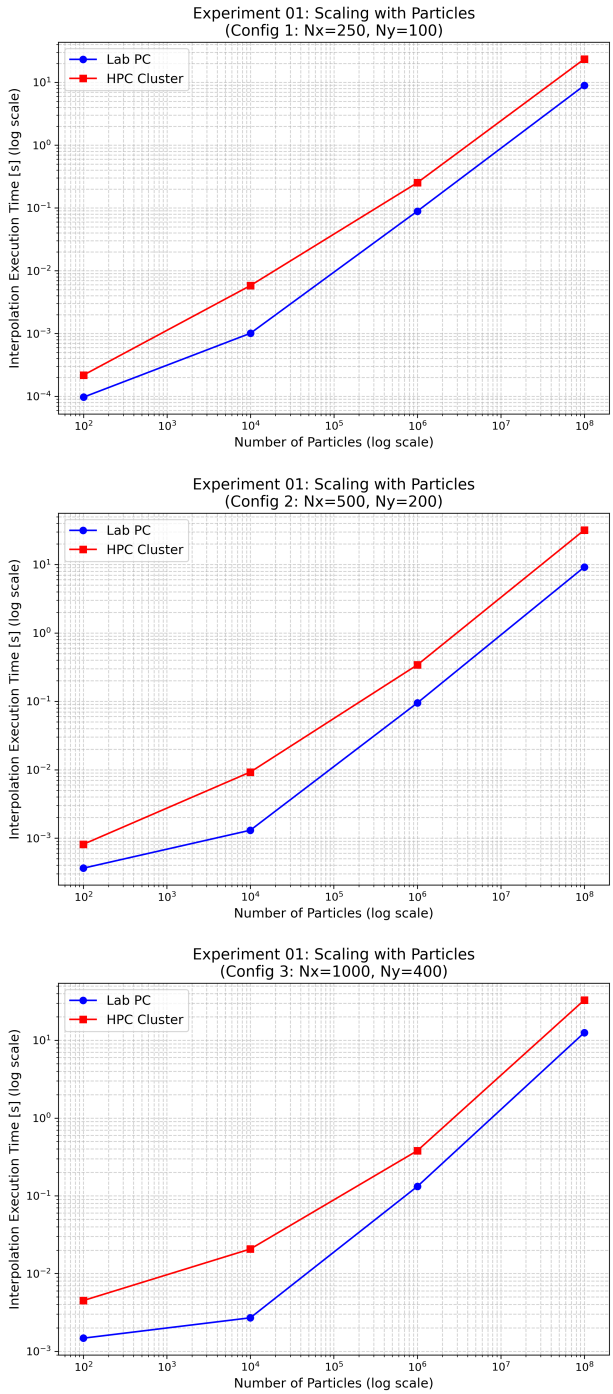


Figure 1: Log-log plots of execution time vs number of particles for Config 1, 2, and 3 comparing Lab PC and HPC cluster.

5.2 Experiment 02 Plots

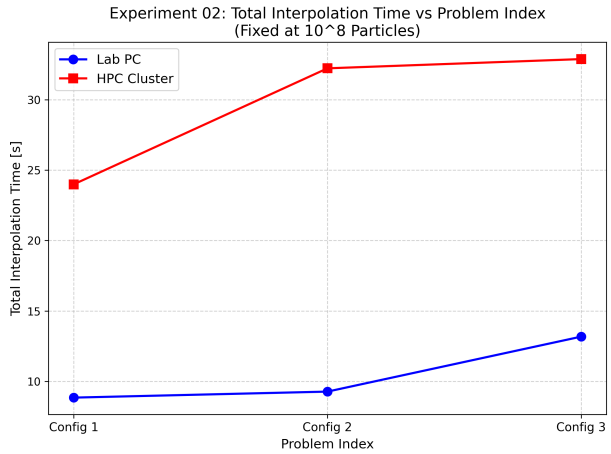


Figure 2: Total interpolation time vs Problem Index (1 to 3) for Lab PC and HPC cluster.

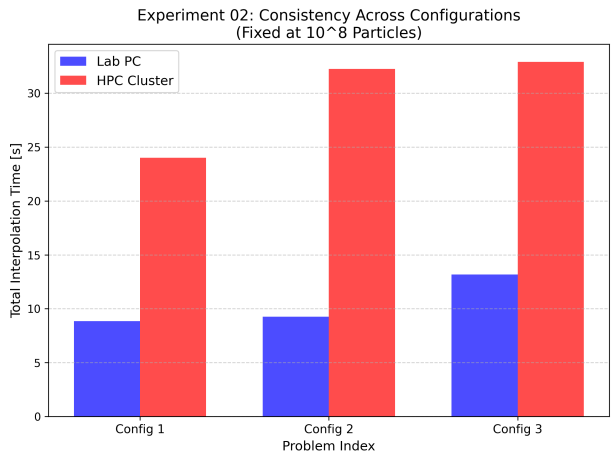


Figure 3: Bar plot of execution time vs. problem index for Lab PC and HPC cluster.

5.3 Experiment 03 Plots

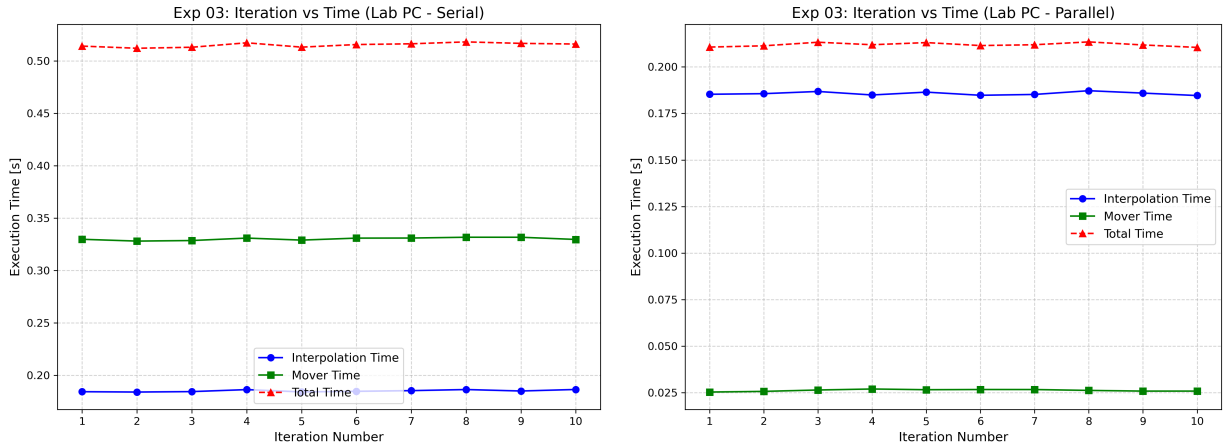


Figure 4: Iteration vs. Time for Lab PC (Left: Serial, Right: Parallel).

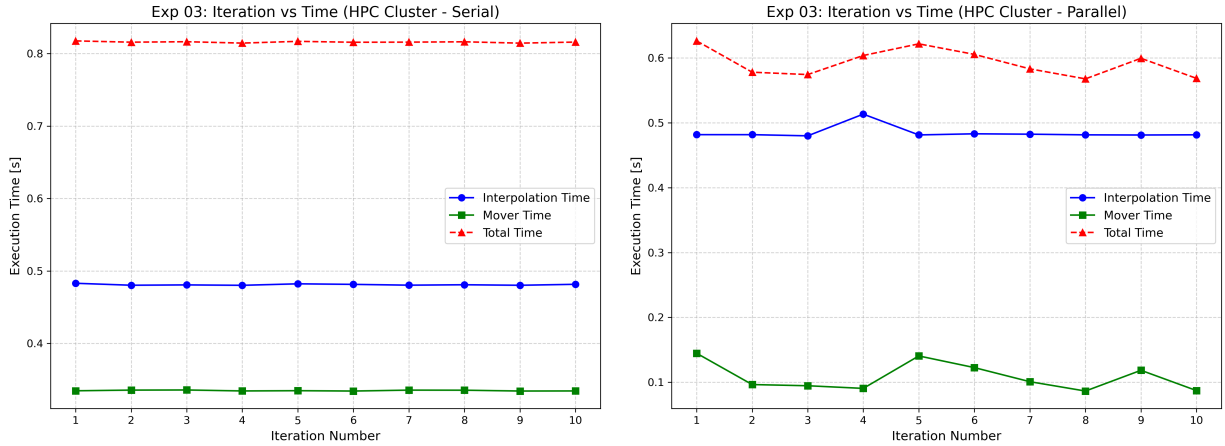


Figure 5: Iteration vs. Time for HPC Cluster (Left: Serial, Right: Parallel).

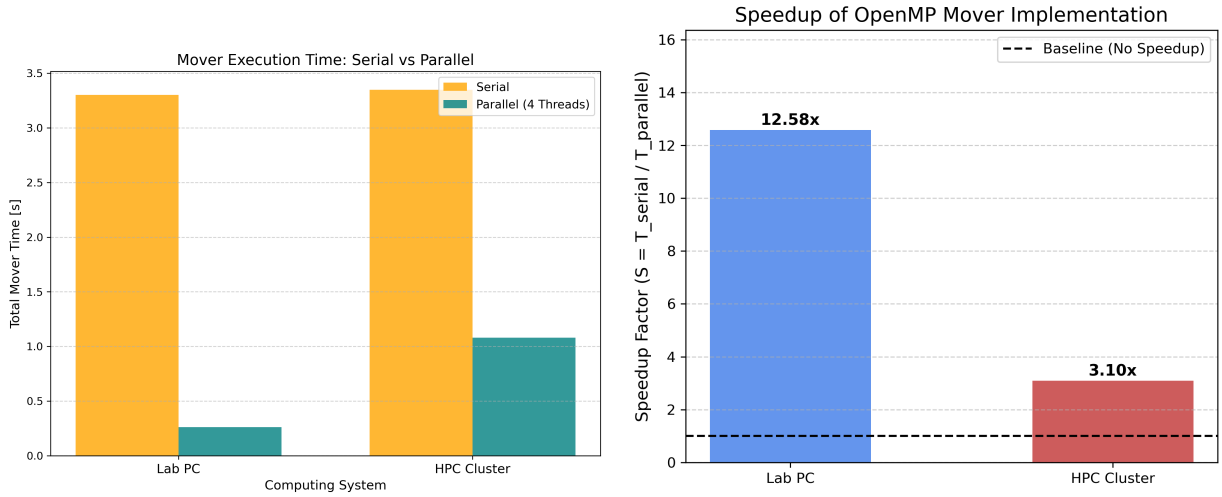
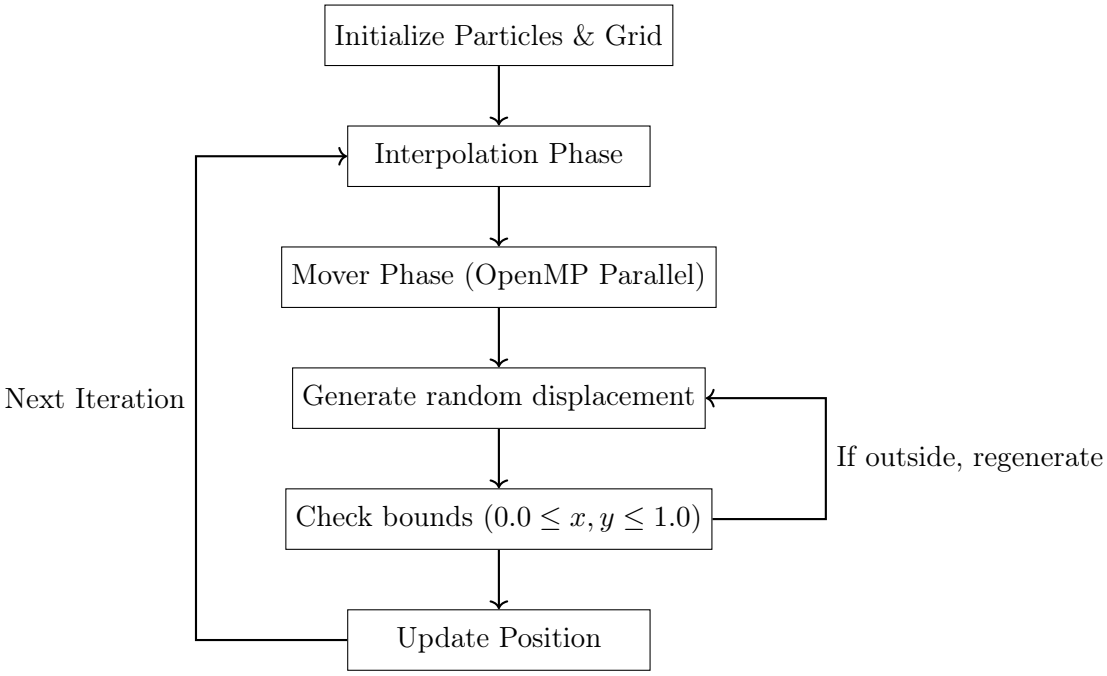


Figure 6: (Left) Total Mover execution times comparing serial and parallel implementations. (Right) Speedup factor achieved by the OpenMP parallel Mover implementation.

6 Implementation Approach

6.1 Schematic Flow Diagram



7 Analysis and Questions

Q1. **Include all required plots from Experiments 01, 02, and 03.**

All corresponding plots have been attached in Section 5.

Q2. **Create a bar plot of execution time vs. problem index for both the Lab PC and the HPC cluster.**

The requested bar plot is presented as Figure 3 in Section 5.2.

Q3. **Explain your overall approach pictorially in the report using a schematic or flow diagram.**

Refer to Section 6 for the schematic diagram encompassing the Interpolation and OpenMP Mover phases.

Q4. **Comment on the observed differences in performance between the Lab PC and the HPC cluster, and explain the reasons based on architectural characteristics.**

For purely serial execution, the Lab PC significantly outperforms the HPC cluster. This is driven by architectural characteristics: the Lab PC utilizes an Intel i5-12500 with a max clock speed of 4.60 GHz, whereas the HPC cluster relies on an older Intel Xeon E5-2640 with a slower clock speed of 2.60 GHz. The HPC cluster is optimized for heavily multi-threaded, parallel operations rather than single-core serial execution.

Q5. **Calculate the speedup achieved by the OpenMP parallelization of the Mover operation and discuss your observations in relation to hardware properties.**

Speedup is calculated as $S = T_{serial}/T_{parallel}$. Based on the results in Section 5.3, the HPC Cluster achieved a speedup of **3.10x**, which closely aligns with expectations for 4 threads (the small gap is due to OpenMP overhead and memory bandwidth constraints).

Interestingly, the Lab PC achieved a super-linear speedup of **12.58x**. Since we are strictly using 4 threads, this high speedup implies additional architectural benefits triggered by OpenMP. The Lab PC's more modern i5-12500 likely benefits from auto-vectorization (SIMD) enabled by the compiler during parallel execution, or the segmented workload fits much more efficiently into the massive 18MB L3 cache, severely reducing memory latency.

Q6. **If you were asked to parallelize the Interpolation function, what would be your approach? Would there be any problems or bottlenecks while parallelizing the interpolation function?**

If parallelizing the interpolation step, I would distribute the particles across the OpenMP threads. **Bottlenecks:** A major problem arises when multiple scattered points fall within the same or adjacent cells and attempt to update shared grid points simultaneously. This results in race conditions. Fixing this requires using atomic operations (which severely impacts performance due to hardware locking overhead) or giving each thread a private copy of the grid to reduce at the end (which heavily increases memory usage).

8 Conclusions

- **Scaling Behavior:** The performance of the interpolation algorithm scales linearly with the number of particles across all tested grid configurations, adhering to the expected theoretical time complexity.
- **Iteration Consistency:** Consistency evaluations demonstrate that when initialization is performed only once, the interpolation execution time remains highly stable across all iterations.

- **Parallel Mover Implementation:** The Mover phase introduces significant computational cost due to random number generation and bounds checking. Parallelizing this operation using OpenMP with 4 threads successfully yields a significant performance boost.
- **Hardware Variations:** The HPC cluster shows a predictable sub-linear speedup (3.10x for 4 threads). Conversely, the Lab PC demonstrates super-linear scaling (12.58x), showcasing how modern CPUs with large caches and advanced vectorization capabilities interact with OpenMP workloads.